P0418r2
Fail or succeed: there is no atomic lattice

Published Proposal, 9 November 2016

This version:
http://wg21.link/p0418r2

Authors:
JF Bastien (Apple)
Hans Boehm (Google)

Audience:
SG1, LWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0418r2.bs

Abstract

Try to resolve [LWG2445].

Table of Contents

§ 1. Background

[LWG2445] was discussed and resolved by SG1 in Urbana.

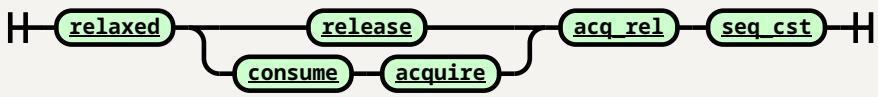
This revision updates [P0418r1] with accurate wording for [util.smartptr.shared.atomic] ¶32, to be deleted from [N4606].

§ 1.1. LWG issue #2445

> The definitions of compare and exchange in [util.smartptr.shared.atomic] ¶32 and [atomics.types.operations.req] ¶21 state:
>
> > Requires: The failure argument shall not be `memory_order_release` nor `memory_order_acq_rel`. The failure argument shall be no stronger than the success argument.
>
> The term "stronger" isn't defined by the standard.
>
> It is hinted at by [atomics.types.operations.req] ¶22:
>
> > When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.
>
> Should the standard define a partial ordering for memory orders, where consume and acquire are incomparable with release?

1.2. Proposed SG1 resolution from Urbana

Add the following note:

> [*Note:* Memory orders have the following relative strengths implied by their definitions:
>
> 
>
>
> *—end note*]

1.3. Further issue

Nonetheless:

- The resolution isn't on the LWG tracker.
- The proposed note was never moved to the draft Standard.

Furthermore, the resolution which SG1 came to in Urbana resolves what "stronger" means by specifying a lattice, but isn't not clear on what "The failure argument shall be no stronger than the success argument" means given the lattice.

There is no relationship, "stronger" or otherwise, between release and consume/acquire. The current wording says "shall be no stronger" which isn't the same as "shall not be stronger" in this context. Is that on purpose? At a minimum it's not clear and should be clarified.

Should the following be valid:

```
compare_exchange_strong(x, y, z, memory_order_release, memory_order_acquire);
```

Or does the code need to be:

```
compare_exchange_strong(x, y, z, memory_order_acq_rel, memory_order_acquire);
```

Similar questions can be asked for `memory_order_consume` ordering on `failure`.

Is there even a point in restricting `success`/`failure` orderings? On architectures with load-linked/store-conditional instructions the load and store are distinct instructions which can each have their own memory ordering (with appropriate leading/trailing fences if required), whereas architectures with compare-and-exchange already have a limited set of instructions to choose from. The current limitation (assuming [LWG2445] is resolved) only seems to restrict compilers on load-linked/store-conditional architectures.

The following code could be valid if the stored data didn't need to be published nor ordered, whereas any retry needs to read additional data:

```
compare_exchange_strong(x, y, z, memory_order_relaxed, memory_order_acquire);
```

Even if—for lack of clever instruction—architectures cannot take advantage of such code, compiler are able to optimize atomics in all sorts of clever ways as discussed in [N4455].

2. Updated proposal

This paper proposes removing the "stronger" restrictions between compare-exchange's `success` and `failure` ordering, and doesn't add a lattice to order atomic orderings. The only remaining restriction is that `memory_order_release` and `memory_order_acq_rel` for `failure` are still disallowed: a failed compare-exchange doesn't store, the current model is therefore not sensible with these orderings.

There have been discussions about `memory_order_release` loads, e.g. for seqlock. Such potential changes are left up to future papers.

Modify [util.smartptr.shared.atomic] ¶32 as follows:

> Requires: The failure argument shall not be `memory_order_release` ~~,~~ nor `memory_order_acq_rel` ~~, or stronger than success~~ .

Modify [atomics.types.operations.req] ¶21 as follows:

> Requires: The failure argument shall not be `memory_order_release` nor `memory_order_acq_rel`. ~~The failure argument shall be no stronger than the success argument.~~

Leave [atomics.types.operations.req] ¶22 as-is:

> Effects: Atomically, compares the contents of the memory pointed to by `object` or by `this` for equality with that in `expected`, and if `true`, replaces the contents of the memory pointed to by `object` or by `this` with that in `desired`, and if `false`, updates the contents of the memory in `expected` with the contents of the memory pointed to by `object` or by `this`. Further, if the comparison is `true`, memory is affected according to the value of `success`, and if the comparison is `false`, memory is affected according to the value of `failure`.
>
> When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.
>
> If the operation returns `true`, these operations are atomic read-modify-write operations (1.10). Otherwise, these operations are atomic load operations.

3. Acknowledgement

Thanks to John McCall for pointing out that the proposed resolution was still insufficient, and for providing ample feedback.

§ References

§ Informative References

[LWG2445]
JF Bastien. "Stronger" memory ordering. SG1. URL: http://cplusplus.github.io/LWG/lwg-active.html#2445

[N4455]
JF Bastien. No Sane Compiler Would Optimize Atomics. 10 April 2015. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4455.html

[N4606]
Richard Smith. Working Draft, Standard for Programming Language C++. 12 July 2016. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4606.pdf

[P0418r1]
JF Bastien. Fail or succeed: there is no atomic lattice. 2 August 2016. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0418r1.html